



UNIVERSITÉ DE PARIS

WEB PROGRAMMING 2026

Spotly

Full-Stack Event Management System

Technical Report

Supervisor: Dr. Alla Jammime

Authors: Lana HUYNH, Yanis SAADI DIT SAADA

Program: Master 1

Academic Year: 2025–2026

Contents

1	Introduction	2
2	Project Requirements and Functional Analysis	2
3	General Project Architecture	2
4	Django Backend Implementation	3
5	Node.js/Express Backend and Comparative Analysis	3
5.1	Implemented Express Backend	4
5.2	Testing Screenshots	4
5.3	Project Structure	4
5.4	Architectural Philosophy	5
5.5	Development Complexity	5
5.6	Scalability	5
5.7	Ecosystem	5
5.8	Comparative Summary	6
5.9	Discussion	6
6	React Frontend Implementation	6
7	Frontend-Backend Integration	6
8	Deployment and Production Configuration	7
9	Challenges Encountered	7
10	Conclusion	7

1. Introduction

We developed an integrated full-stack project called **Spotly**. The goal of this project was to design, implement, and deploy a complete web application for event and participant management, while also integrating a REST API, a modern client-side interface, and a comparative analysis between Node.js and Django.

The application had to demonstrate several technical skills, including relational data modeling, REST API design, Single Page Application development with React, frontend-backend integration, authentication, role management, and production deployment.

Spotly was designed to allow administrators to manage events, participants, and registrations, while viewers can access the data in read-only mode. This project allowed us to learn the development of a modern web application.

2. Project Requirements and Functional Analysis

The project required the implementation of an event and participant management system, including participant registration to events through a dedicated many-to-many relationship using a **Registration** model. This requirement ensures better control over business rules and prevents duplicate registrations.

The main features are the following:

- create, update, delete, and list events;
- filter events by date and status;
- create, update, delete, and list participants;
- register participants to one or several events;
- authentication with at least two roles: staff/editor and viewer;
- a frontend implemented as a React SPA;
- public deployment of both backend and frontend.

The system relies on three main entities:

- **Event**: stores event information such as name, description, date, and status;
- **Participant**: stores participant information;
- **Registration**: links participants and events.

3. General Project Architecture

The project is divided into two main parts: a Django REST Framework backend and a React frontend.

The backend is responsible for managing the models, business rules, JWT authentication, REST endpoints, and API documentation. The frontend consumes this API to display data, manage forms, and restrict access depending on the logged-in user's role.

The architecture is:

- **Backend:** Django, Django REST Framework, SimpleJWT, drf-spectacular
- **Frontend:** React, React Router, hooks, API calls
- **Communication:** REST API secured with JWT tokens
- **Database:** SQLite in development
- **Deployment:** backend on Render, frontend on Vercel

The application uses SQLite. This choice was sufficient to validate the relational model, API integration, and deployment workflow. However, for a larger-scale production environment, PostgreSQL would be a more robust choice because it offers better scalability and reliability.

4. Django Backend Implementation

The backend was developed using **Django 5.2.12** and **Django REST Framework 3.16.1**. Several additional libraries were used to support the project: `django-rest-framework-simplejwt` for authentication, `django-cors-headers` for frontend-backend communication, `drf-spectacular` for OpenAPI documentation, and `gunicorn` and `whitenoise` for deployment preparation.

The API provides important routes:

- business endpoints under `/api/`;
- JWT authentication endpoints under `/api/token/` and `/api/token/refresh/`;
- interactive API documentation under `/api/docs/`.

JWT was used to provide stateless authentication. During login, an access token and a refresh token are generated. The access token is then used by the frontend when calling protected endpoints.

The backend also includes CORS configuration in order to authorize requests from the local frontend as well as the deployed frontend. It was necessary to ensure the integration between React and Django.

Finally, the backend was prepared for production using Gunicorn, allowed hosts configuration, and static files management.

5. Node.js/Express Backend and Comparative Analysis

As part of Lab 8, we developed a simplified backend using Node.js and Express and compared it with the Django backend used in the main application. This section presents both the implemented Express backend and the analytical comparison.

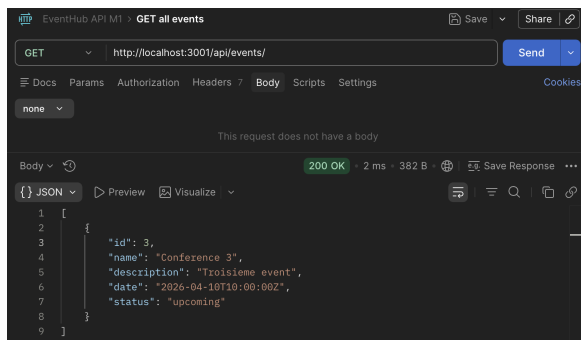
5.1. Implemented Express Backend

The Express backend created for Lab 8 is a simplified version of the Spotly API. In our implementation, we created two entities: **events** and **participants**.

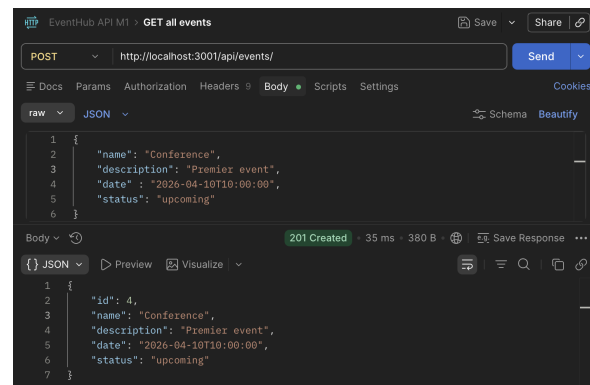
The backend is connected to SQLite and returns JSON responses, which made testing straightforward in Postman. We also added basic middleware such as JSON request parsing, CORS handling, and simple route-level error management.

This implementation showed that Express is efficient for building a lightweight REST API. The code is relatively short and the setup is fast, which makes Node.js/Express a practical choice for rapid prototyping and small backend services. At the same time, the implementation also helped us observe its limits compared with Django.

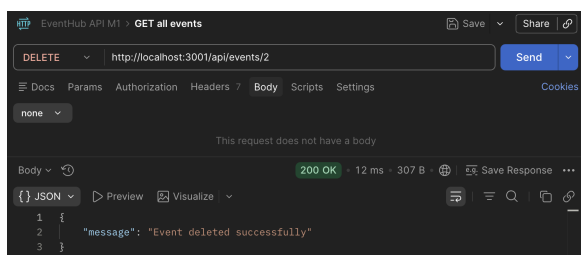
5.2. Testing Screenshots



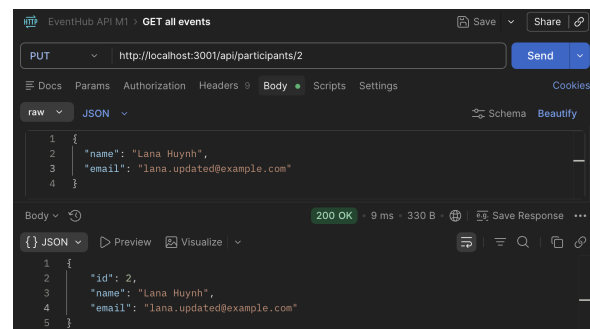
(a) GET /api/events



(b) POST /api/events



(c) DELETE /api/events/2



(d) PUT /api/participants/2

Figure 1: Validation of the main CRUD operations in the Express backend.

5.3. Project Structure

One major difference between Django and Node.js/Express is project structure. Django follows a more organized and convention-based architecture, where models, serializers, views, migrations, and routing are generally separated in a predictable way. This kind of structure is useful in team projects because responsibilities are clearer and the codebase is easier to maintain.

By contrast, Express is much more flexible. It doesn't impose a strict file structure, so developers are free to organize routes, middleware, database logic, and controllers as they prefer. This is convenient but in larger applications it can lead to inconsistent organization.

5.4. Architectural Philosophy

Django provides many built-in tools for backend development, such as an ORM, authentication support, administrative interfaces, and good compatibility with REST API development. Because of this, developers can focus more on application logic and less on assembling technical components.

Express mainly provides a lightweight framework for handling HTTP requests and responses, while most other features depend on external packages and design choices made by the developer. This makes Express very flexible, but also means that more architectural decisions must be taken manually.

5.5. Development Complexity

In terms of development complexity, Express is easier to start with. A basic backend can be created quickly with only a few files, which is why it works well for a simplified API with CRUD routes and a database connection.

Django is more complex at the beginning because it requires understanding several concepts such as applications, models, migrations, serializers, and viewsets. However, once these concepts are learned, development becomes more suitable for applications with relational data, permissions, and business rules.

5.6. Scalability

Both Django and Node.js can scale well. Django is particularly well adapted to applications with structured relational data, clear business logic, and maintainable backend organization. For a project like Spotly, this is important because the system includes several related entities and role-based behavior.

Express also scales well, especially in modular and lightweight service architectures. Its flexibility is great for custom APIs and smaller services where developers want full control over the stack.

5.7. Ecosystem

Django benefits from a mature ecosystem centered on backend web development. It is especially strong for relational databases, authentication, administration interfaces, and REST-oriented applications.

Node.js also has a very large ecosystem. Express can be combined with many external packages for routing, validation, authentication, and database integration, which gives developers freedom. The advantage of this ecosystem is flexibility, but the disadvantage is that developers must choose and integrate more tools by themselves.

5.8. Comparative Summary

Overall, the comparison shows that Django and Node.js/Express are designed for different development styles. Django offers a more structured project organization and a mature ecosystem. This makes it particularly suitable for applications that require strong maintainability and integrated backend features.

Node.js/Express is more lightweight and flexible. It allows developers to build APIs quickly and organize the codebase freely. However, this flexibility also means that more architectural decisions must be taken manually, especially when the project grows in size and complexity.

Finally, both ecosystems are strong, but Django provides more integrated tools out of the box, while Express relies more on external packages and developer choices.

5.9. Discussion

For this project, Django is the more suitable technology for the main backend because the overall application requires a complete relational model, structured API development, and maintainable backend organization.

6. React Frontend Implementation

The assignment required several mandatory pages: Login, Events, Event Details, Participants, and Dashboard. These pages were implemented using React Router and functional components with React Hooks.

The **Login** page allows users to enter their credentials, send an authentication request to the backend, and store the returned tokens after the login. Once logged-in, the user is redirected to the dashboard.

The **Events** page displays the list of events, supports filtering by status and date, and allows staff users to create new events. The **Event Details** page displays detailed information about a given event and the list of registered participants. The **Participants** page allows staff users to view and manage participant profiles.

The frontend also handles loading states and error messages, which improves the overall user experience. Navigation is managed through a dedicated navbar, and access to sensitive pages is restricted through route protection.

Particular attention was given to the distinction between viewers and staff users. Actions such as create, update, and delete are restricted to staff users, while viewers only have read-only access.

7. Frontend-Backend Integration

The integration between React and Django is the main aspect of the project. The frontend directly consumes the REST endpoints exposed by the backend in order to retrieve event and participant lists, display event details, and submit creation forms.

This communication is based on HTTP requests sent from React components to the Django API. The JWT access token is used to authenticate requests whenever necessary. This approach

ensures a clear separation between client and server.

The integration phase also required special attention for:

- handling network errors
- displaying loading states
- token storage
- CORS compatibility
- consistency between backend permissions and frontend rendering

The frontend and backend interact correctly, both in local development and in preparation for production deployment.

8. Deployment and Production Configuration

The assignment required a deployed version of the project. The backend was prepared to run with Gunicorn and configured for deployment on Render. Allowed hosts and CORS origins were configured to support both local development and online access.

The React frontend was tested in production mode using the `npm run build` command, which generates an optimized version of the application.

Moving from development mode to production revealed several important differences, including API URLs, static files configuration, security settings and React Router compatibility.

Although final deployment may still require adjustments depending on the hosting platform, the project was designed with real production deployment in mind.

9. Challenges Encountered

Several technical challenges were encountered during development. The first major issue involved authentication and role management. It was necessary not only to secure the backend API, but also to adapt the frontend interface depending on the role of the connected user.

The second difficulty concerned the integration between the different entities. The many-to-many relationship between events and participants, implemented through the `Registration` model, required more advanced logic than a simple direct association.

Another important challenge involved the frontend, especially the correct use of React Hooks, the organization of components, and the conditional rendering of actions depending on the user role. This part required several corrections before obtaining a coherent implementation.

10. Conclusion

This project allowed us to build a coherent full-stack application while integrating the main concepts expected in a web development project: relational modeling, REST APIs, React SPA development, authentication, role management, multi-technology integration, and deployment.

The chosen combination of Django REST Framework for the backend and React for the frontend proved to be effective. Django enabled rapid and structured API development, while React provided a modern and modular user interface.